# Veri Yapıları

## Bağlı Listeler

Dr. Öğr. Üyesi Muhammed Ali KOŞAN

# İçerik-Teorik

- Liste ADT’si ve Dizi (Array)
- Tek Yönlü Bağlı Listeler (Singly Linked List)
- Çift Yönlü Bağlı Listeler (Doubly Linked List)
- Dairesel Bağlı Listeler (Circular Linked List)
- Gerçek Dünya Uygulamaları

# Soyut Veri Tipi (ADT) Nedir?

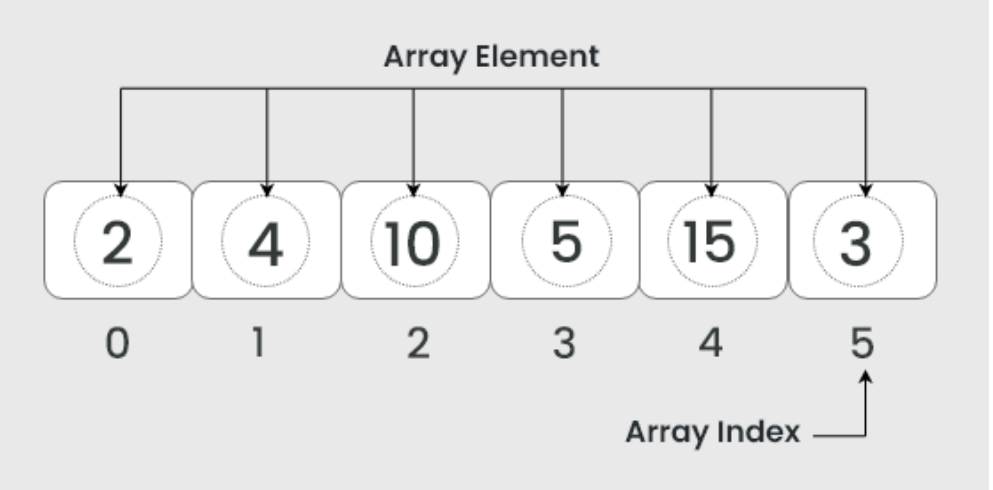
- **Tanım**
  - Bir veri tipinin ne yaptığını (operasyonlarını) tanımlayan, ancak nasıl yaptığı (implementasyonu) ile ilgilenmeyen bir modeldir.
- **Yazılım Mühendisliği İlkesi**
  - Arayüz (Interface) vs. Implementasyon
- **Örnek → Liste (List) ADT'si**
  - **Veri**
    - Sıralı bir eleman koleksiyonu ($A_1$, $A_2$, …, $A_N$).
  - **Operasyonlar**
    - insert(x, pos), remove(pos), find(x), get(pos), size().

# Implementasyon → Diziler / std::vector

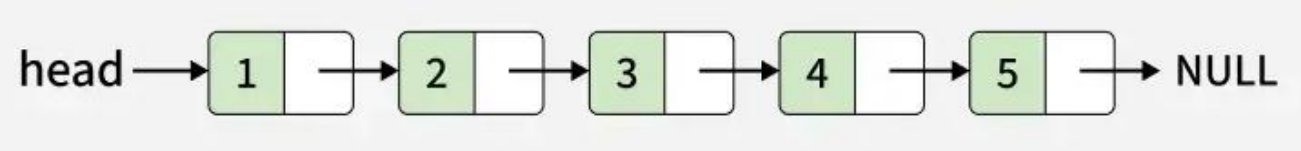
- Veriler bellekte ardışıl (contiguous) olarak saklanır.
- **Operasyonların Big-O Analizi**
  - get(i) (Indeks ile Erişim)
    - O(1) → Mükemmel! (Adresi anında biliriz)
  - find(x) (Değer Arama)
    - O(n)
  - push_back(x) (Sona Ekleme)
    - O(1) (Amortized)
  - insert(x, position 0) (Başa Ekleme)
    - O(n)
  - remove(position 0) (Baştan Silme)
    - O(n)

# Dizilerin (Array) Zayıf Yönü

- **Problem**
  - Başa veya ortaya ekleme/silme işlemi, listenin geri kalan tüm elemanlarının bellekte kaydırılmasını (shifting) gerektirir.
- **Sonuç**
  - Diziler dinamik değildir (boyutları sabittir veya yeniden boyutlandırma O(n) maliyetlidir) ve ekleme/silme işlemleri verimsizdir.
- **Soru**
  - O(1) zamanda ekleme/silme yapabilir miyiz?
- **Çözüm**
  - Bağlı Listeler



# Bağlı Listeler (Linked Lists)




- **Ana Fikir**
  - Verileri bellekte dağınık tut. Her veri parçası (Node), kendisinden sonra gelen veri parçasının bellek adresini (next pointer) tutsun.
- **Yapı**
  - Düğümler (Nodes) ve bir head (başlangıç) işaretçisinden oluşur.

```cpp
class Node {
public:
    int data;
    Node* next;

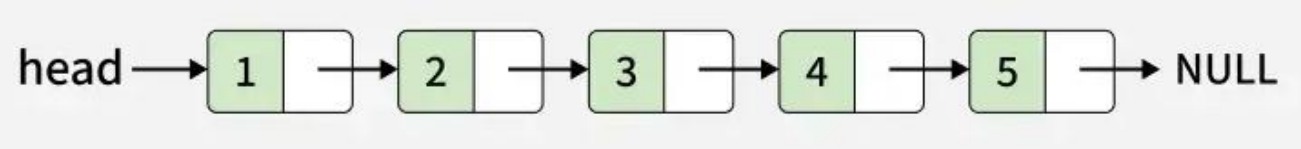
    Node(int val) {
        data = val;
        next = nullptr;
    }
};
```

```cpp
Node* head = new Node(1);
head->next = new Node(2);
head->next->next = new Node(3);
head->next->next->next = new Node(4);
head->next->next->next->next = new Node(5);

while (head != nullptr) {
    cout << head->data << " ";
    head = head->next;
}
```

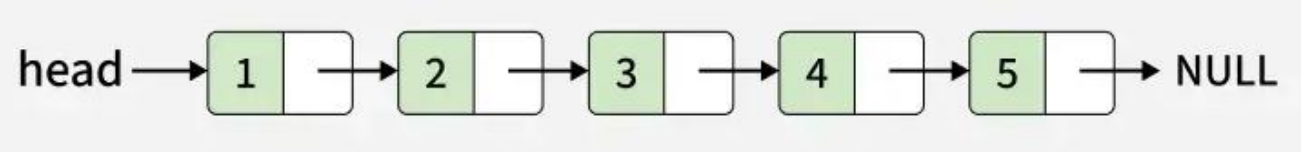
# Temel Operasyonlar ve Analizi

- **Dolaşma (Traversal) / *printList()***
  - Geçici bir Node* temp = head; ile başla.
  - temp != nullptr olduğu sürece temp->data'yı yazdır ve temp = temp->next; ile ilerle.
  - Karmaşıklık → O(n)
- **Arama (Search)**
  - Dolaşma ile aynıdır. Aranan data bulununca durulur.
  - Karmaşıklık → O(n) (En kötü durum).



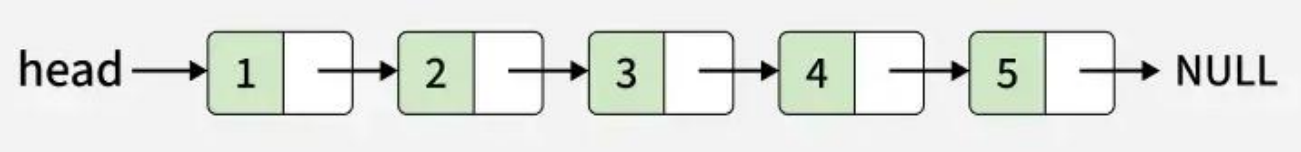
# Ekleme (Insertion) Operasyonları

- **Başa Ekleme (push_front)**
  - Karmaşıklık → O(1) (Dizide O(n) idi!)
- **Sona Ekleme (push_back)**
  - Listeyi O(n)'de dolaşarak son düğümü (last) bul.
  - last->next = newNode;
  - Karmaşıklık → O(n)
  - İyileştirme → Eğer LinkedList sınıfı tail (kuyruk) işaretçisi de tutarsa, bu işlem O(1) olur.
- **Araya Ekleme (insert_after(prev_node, data))**
  - prev_node->next'i kaybetmemek önemlidir.
  - newNode->next = prev_node->next;
  - prev_node->next = newNode;
  - Karmaşıklık → O(1) (Eğer prev_node'un adresi biliniyorsa)



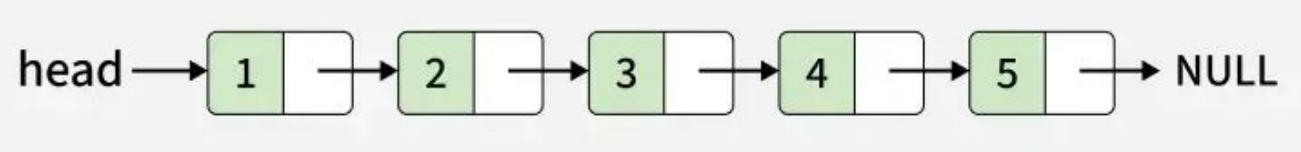
# Silme (Deletion) Operasyonları

- **Baştan Silme (pop_front)**
  - Node* temp = head;
  - head = head->next;
  - delete temp; (Bellek sızıntısını önle!)
  - Karmaşıklık → O(1)
- **Belirli Bir Değeri Silme (deleteNode(key))**
  - Silinecek düğümü (curr) ve ondan önceki düğümü (prev) bulmak için listeyi dolaş (prev'i tutmak zorunludur!).
  - prev->next = curr->next;(Aradaki curr atlanır)
  - delete curr; (Bellek serbest bırakılır)
  - Karmaşıklık → O(n) (Arama maliyeti).



# Bellek Yönetimi ve Yıkıcı Metot

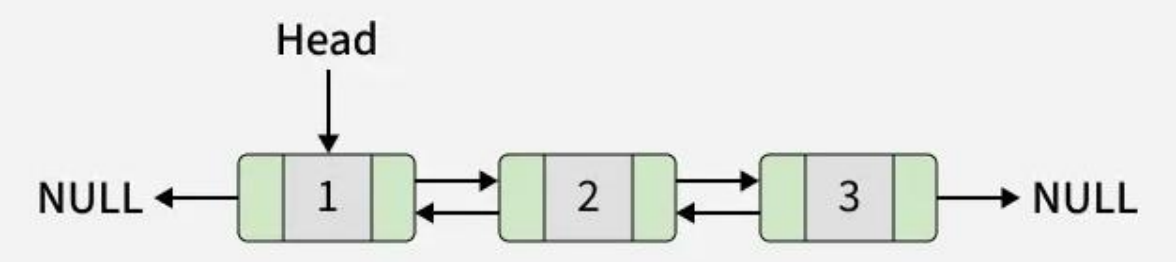
- **Problem**
  - LinkedList nesnesi yok edildiğinde, new ile oluşturulan tüm Node'lar bellekte kalır → Bellek Sızıntısı (Memory Leak).
- **Çözüm**
  - Yıkıcı Metot yazılmalıdır.
- **Algoritma**
  - Liste boşalana kadar pop_front() işlemini tekrarla veya current ile dolaşırken her düğümü delete et.

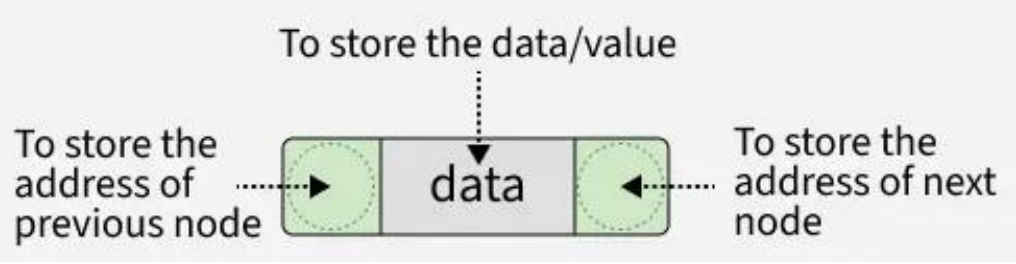


# Çift Yönlü Bağlı Listeler (Doubly Linked List)

- **Problem**
  - Tek Yönlü Listede geriye gidemeyiz. Bir düğümü silmek için O(n)'de ondan öncekini bulmak gerekir.
- **Çözüm**
  - Her düğüm hem next hem de prev (önceki) işaretçisini tutar.
- **Avantaj**
  - Çift yönlü dolaşma.
  - Bir düğümün adresi biliniyorsa, silme işlemi O(1)'dir (çünkü curr->prev'e anında erişilebilir).
- **Dezavantaj**
  - Daha fazla bellek (her düğüm için +1 pointer).
  - Ekleme/Silme daha karmaşık (4 işaretçi güncellenmeli).

# Çift Yönlü Bağlı Listeler (Doubly Linked List)




```
class Node {
public:
    int data;
    Node* prev;
    Node* next;

    Node(int d) {
        data = d;
        prev = nullptr;
        next = nullptr;
    }
};
```

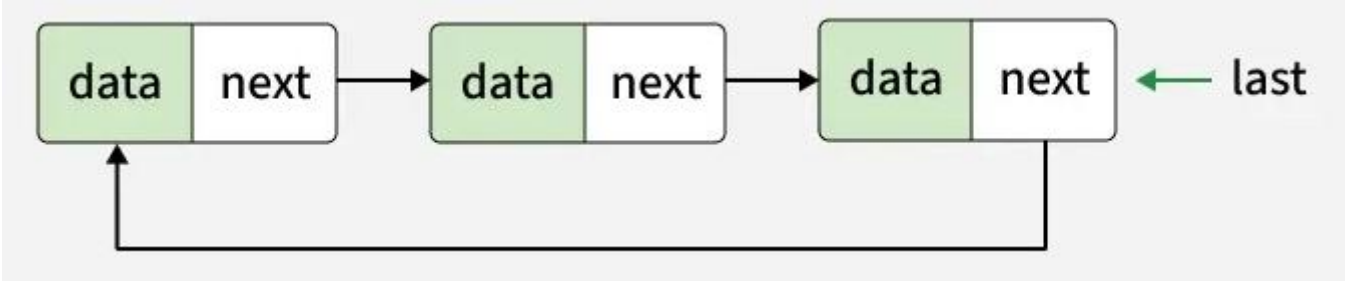
```
Node* head = new Node(10);
head->next = new Node(20);
head->next->prev = head;
head->next->next = new Node(30);
head->next->next->prev = head->next;
head->next->next->next = new Node(40);
head->next->next->next->prev = head->next->next;

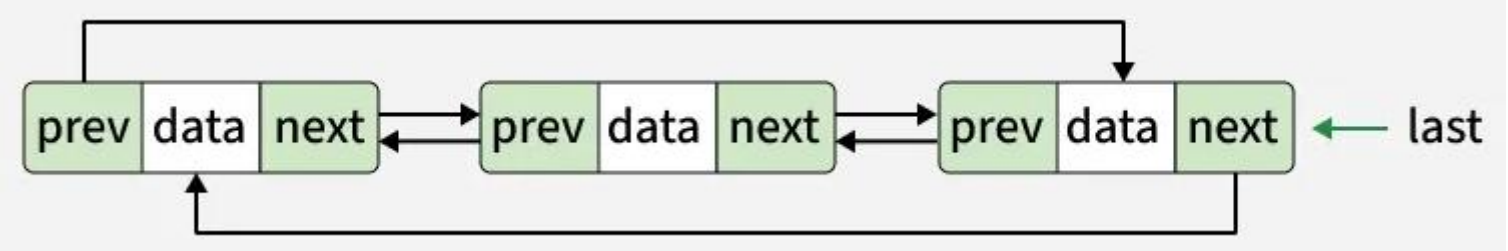
Node* temp = head;
while (temp != nullptr) {
    cout << temp->data;
    if (temp->next != nullptr) {
        cout << " <-> ";
    }
    temp = temp->next;
}
```

# Dairesel Bağlı Listeler (Circular Linked List)

- **Tanım**
  - Son düğümün next işaretçisi nullptr yerine head'i (veya tail'i) gösterir.
- **Avantajlar**
  - Liste "döner". Herhangi bir düğümden başlayarak tüm liste gezilebilir.
  - Round-Robin (dönüşümlü) algoritmalar için idealdir (Örn. İşletim sistemi işlem zamanlaması).
  - **Verimli Kuyruk (Queue) implementasyonu**
    - Sadece tail işaretçisi tutularak head'e (tail->next) ve tail'e O(1)'de erişilebilir.
- **Önemli!**
  - Dolaşma (Traversal) yapılırken sonsuz döngüye girmemek için temp != head (veya do-while) kontrolü yapılmalıdır.

# Dairesel Bağlı Listeler (Circular Linked List)



# Karşılaştırma Tablosu

| Operasyon | std::vector (Dizi) | Tek Yönlü (Singly) | Çift Yönlü (Doubly) |
|---|---|---|---|
| **İndeksle Erişim (get(i))** | **O(1)** | O(n) | O(n) |
| **Arama (find(x))** | O(n) | O(n) | O(n) |
| **Başa Ekleme/Silme** | O(n) | **O(1)** | **O(1)** |
| **Sona Ekleme/Silme** | O(1) (Amortized) | O(1) (Eğer tail varsa) | **O(1)** (Eğer tail varsa) |
| **Bilinen Düğümü Silme** | - | O(n) (Öncekini bul) | **O(1)** |
| **Bellek Yükü** | Düşük | n x (pointer) | 2n x (pointer) |

# Gerçek Dünya Uygulamaları

- **Tek Yönlü Listeler**
  - Stack ve Queue (Kuyruk) veri yapılarının temel implementasyonu.
  - Grafların (Graph) Bitişiklik Listesi (Adjacency List) gösterimi.
- **Çift Yönlü Listeler**
  - Web tarayıcılarda "İleri" ve "Geri" butonları.
  - Metin editörlerinde (Notepad vb.) "Geri Al" (Undo) ve "Yinele" (Redo) fonksiyonları.
  - Müzik/Video oynatıcılarda "Önceki Şarkı / Sonraki Şarkı" listesi.
- **Dairesel Listeler**
  - İşletim sistemlerinde Round-Robin zamanlama (CPU'nun işlemlere sırayla zaman ayırması).
  - Müzik çaların "Tekrarla" (Repeat) özelliği.

# Özet

- Liste ADT'sini, Dizi ve Bağlı Liste implementasyonlarını öğrendik.
- Tek Yönlü, Çift Yönlü ve Dairesel listelerin farklarını ve Big-O analizlerini yaptık.

# Gelecek Hafta

- Yığın (Stack) yapılarını göreceğiz.
- Devamında Kuyruk (Queue) yapılarını anlayacağız.

# Soru & Cevap

Dr. Öğr. Üyesi Muhammed Ali KOŞAN

# Veri Yapıları

## Bağlı Listeler
## (Uygulama Dersi)

Dr. Öğr. Üyesi Muhammed Ali KOŞAN

# İçerik-Uygulama

- Temel Tek Yönlü Bağlı Liste Sınıfı
- Ekleme, Silme ve Bellek Yönetimi

# Uygulama:
# Tek Yönlü Bağlı Liste Sınıfı

- **Amaç** → Sıfırdan bir SinglyLinkedList sınıfı yazmak.
- **Görevler**
  - Node sınıfını/struct'ını oluşturun.
  - SinglyLinkedList sınıfını oluşturun (head ve tail işaretçileri ile).
  - SinglyLinkedList() (Yapıcı) ve ~SinglyLinkedList() (Yıkıcı) metotlarını oluşturun.
  - void push_front(int data)
    - Başa O(1) zamanda ekleme.
  - void push_back(int data
    - Sona O(1) zamanda ekleme (tail kullanarak).
  - void printList()
    - Listeyi yazdırma.
  - **Test**
    - main() içinde 30 (başa), 40 (sona), 20 (başa) ekleyin. printList() çağırın.
    - **Çıktı**
      - 20 -> 30 -> 40 -> NULL

# Uygulama:
# Tek Yönlü Bağlı Liste Sınıfı

```cpp
struct Node {
    int data;
    Node* next;

    Node(int d) : data(d), next(nullptr) {}
};
```

```cpp
class SinglyLinkedList {
private:
    Node* head;
    Node* tail;

public:
    SinglyLinkedList(){...}

    ~SinglyLinkedList(){...}

    void push_front(int data){...}

    void push_back(int data){...}

    void printList(){...}
};
```

Detaylar sonraki slaytta

```cpp
int main() {
    SinglyLinkedList list;

    list.push_front( data: 30);
    list.push_back( data: 40);
    list.push_front( data: 20);

    list.printList();

    return 0;
}
```

# Uygulama:
# Tek Yönlü Bağlı Liste Sınıfı

```
SinglyLinkedList() : head(nullptr), tail(nullptr) {
}

~SinglyLinkedList() {
    std::cout << "SinglyLinkedList::~SinglyLinkedList()" << std::endl;
}
```

```
void printList() {
    Node* temp = head;
    while (temp != nullptr) {
        std::cout << temp->data << " -> ";
        temp = temp->next;
    }
    std::cout << "NULL" << std::endl;
}
```

```
void push_front(int data) {
    Node* newNode = new Node(data);
    newNode->next = head;
    head = newNode;

    if (tail == nullptr) {
        tail = head;
    }
}

void push_back(int data) {
    Node* newNode = new Node(data);

    if (head == nullptr) {
        head = newNode;
        tail = newNode;
    } else {
        tail->next = newNode;
        tail = newNode;
    }
}
```

# Uygulama:
# Ekleme, Silme ve Bellek Yönetimi

- **Amaç** → Listenin ortasıyla çalışmak ve bellek sızıntılarını önlemek.
- **Görevler (Mevcut sınıfa eklenecek)**
  - **void insert_after(Node* prev_node, int data)**
    - Verilen bir düğümden sonraya ekleme.
    - **void deleteNode(int key)**
      - Verilen key (değer) sahip ilk düğümü silme (Slayt 10'daki gibi prev ve curr takibi gerektirir).
    - **Yıkıcı Metodu Doldurma**
      - ~SinglyLinkedList() metodunun içini clear() mantığıyla doldurun. (Tüm düğümleri delete edin).

# Uygulama:
# Ekleme, Silme ve Bellek Yönetimi

```
Node* getHead() { return head; }
Node* getTail() { return tail; }
```

```
~SinglyLinkedList() {
    Node* current = head;
    while (current != nullptr) {
        Node* next = current->next;
        delete current;
        current = next;
    }
    std::cout << "\n(Liste hafizadan temizlendi.)" << std::endl;
}
void insert_after(Node* prev_node, int data) {
    if (prev_node == nullptr) {
        std::cout << "Hata: 'prev_node' NULL olamaz. Ekleme yapilamadi." << std::endl;
        return;
    }

    Node* newNode = new Node(data);
    newNode->next = prev_node->next;
    prev_node->next = newNode;

    if (prev_node == tail) {
        tail = newNode;
    }
}
```

```
void deleteNode(int key) {
    Node* temp = head;
    Node* prev = nullptr;

    if (temp != nullptr && temp->data == key) {
        head = temp->next;
        if (temp == tail) {
            tail = nullptr;
        }
        delete temp;
        return;
    }

    while (temp != nullptr && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == nullptr) {
        std::cout << "Bilgi: " << key << " degeri listede bulunamadi." << std::endl;
        return;
    }

    prev->next = temp->next;

    if (temp == tail) {
        tail = prev;
    }

    delete temp;
}
```

# Uygulama:
# Ekleme, Silme ve Bellek Yönetimi

```
int main() {
    SinglyLinkedList list;
    list.push_front( data: 30);
    list.push_back( data: 40);
    list.push_front( data: 20);
    std::cout << "30 öne, 40 geriye, 20 öne sırası ile ekleniyor..." << std::endl;
    list.printList();

    Node* node_30 = list.getHead() ? list.getHead()->next : nullptr;
    if (node_30) {
        std::cout << "30'dan sonra 35 ekleniyor..." << std::endl;
        list.insert_after(node_30,  data: 35);
        list.printList();
    }

    std::cout << "Kuyruktan (40) sonra 50 ekleniyor..." << std::endl;
    list.insert_after( prev_node: list.getTail(),  data: 50);
    list.printList();

    std::cout << "\nOrtadan '35' siliniyor..." << std::endl;
    list.deleteNode( key: 35);
    list.printList();

    std::cout << "Bastan '20' siliniyor..." << std::endl;
    list.deleteNode( key: 20);
    list.printList();
```

```
    std::cout << "Sondan '50' siliniyor..." << std::endl;
    list.deleteNode( key: 50);
    list.printList();

    std::cout << "Olmayan '100' silinmeye calisiliyor..." << std::endl;
    list.deleteNode( key: 100);
    list.printList();

    std::cout << "\nKalan son iki eleman da siliniyor..." << std::endl;
    list.deleteNode( key: 30);
    list.deleteNode( key: 40);
    list.printList();

    return 0;
}
```

# Soru & Cevap

Dr. Öğr. Üyesi Muhammed Ali KOŞAN